测试策略与质量保障体系

从架构图到可验证的质量承诺

质量不是最后测出来的，而是在架构边界、依赖关系和交付门禁中被设计出来的。

本讲问题

架构图画完以后，怎样证明它真的满足质量目标？

本讲产出

测试计划 V1、质量门禁清单、最难测试场景说明

课程连接

承接分布式架构视图，进入发布交付前的质量验证

课程链条

从架构设计到质量验证

上一讲回答系统怎样组织，本讲回答这些组织方式怎样被验证。

已完成

服务拆分、接口关系、数据一致性、部署视图

→

本讲追问

这些边界、依赖和质量目标能否被测试证明？

→

下一步

通过质量门禁的系统，才进入发布、运维和回滚设计

课堂连接

预约服务、审批服务、通知服务分开后，接口和故障模式也必须能被验证。

本页动作

课堂提问：你们组上一讲画出的架构图，最难验证的是哪一块？

M4 产出：测试计划 V1 与质量门禁清单

测试计划不是测试用例堆叠，而是说明风险在哪里、用哪一层测试验证、什么标准算通过。

测试计划 V1

范围、层级分配、环境、关键风险、AI辅助边界

质量门禁清单

代码层、构建层、发布层的可量化通过标准

最难测试场景

写清楚难在哪里，是架构耦合还是测试方法选择不当

案例

并发预约、通知降级、学生越权访问，是本讲贯穿示例。

课堂锚点

考试锚点：给定项目，设计三层质量门禁。

导入问题：你的架构能被验证吗？

一个架构如果难以验证，就很难说它已经完成了设计。

定位问题

预约冲突有 Bug，能在 5 分钟内定位到哪一层吗？

故障问题

通知服务挂掉时，预约流程是否还能完成？

安全问题

学生访问管理员接口，权限层是否一定拦住？

判断动作

课堂投票：这三个问题更像测试问题，还是架构问题？

导入

从质量属性场景反向追问测试方案

质量属性写成指标以后，还必须能转化为可执行的验证方案。

场景

100 个学生同时提交预约

指标

P95 响应时间 ≤ 2 秒，重复占用数 = 0

追问

在哪一层测？需要什么环境？失败后如何定位？

案例

第4讲质量属性场景表补充第五列“测试方案”。

课堂锚点

案例题：把性能/并发质量属性转成测试方案。

核心判断：质量是被设计出来的

测试发现质量问题，架构决定质量问题能否被低成本发现和定位。

发现

测试能暴露缺陷，但不能自动降低修复成本

定位

清晰边界让失败结果能指向具体模块

预防

质量门禁把质量要求变成不可跳过的工程约束

案例

同样是通知失败，接口注入架构可以 Mock，硬编码 HTTP 架构必须启动真实服务。

课堂锚点

简答题：为什么测试难写通常是架构耦合的信号？

可测试性：能被高效、精准地验证

可测试性衡量的不是有没有测试框架，而是系统结构是否支持低成本验证。

定义

系统在多大程度上能被高效、精准地验证

高效

测试运行快、环境依赖少、反馈及时

精准

失败结果能定位到函数、模块、接口或外部依赖

案例

同一条预约规则，若必须启动全系统才能测，说明可测试性差。

课堂锚点

名词解释：可测试性 Testability。

可测试性的四个架构条件

边界清晰、依赖可替换、状态可观测、输入输出可控，是测试能写好的前提。

清晰边界

业务逻辑、数据访问、外部调用不混在一起

可替换依赖

数据库、通知、第三方服务可以用 Mock/Stub 替换

可观测状态

日志、返回值、数据库记录、事件都能被检查

可控输入输出

时间、随机数、并发条件可以被测试控制

反面代码：业务逻辑与外部依赖耦合

当业务逻辑直接访问数据库和外部 HTTP 服务，单元测试会被迫变成小型集成测试。

```
class ReservationService:
    def create_reservation(self, student_id, device_id,
timeslot):
        device = db.execute("SELECT * FROM devices WHERE
id=?", device_id)
        requests.post("http://notification-service/send",
...)
```

直接 DB

测试必须准备真实表结构和测试数据

直接 HTTP

通知服务不可用时，预约逻辑测试也会失败

定位困难

失败可能来自业务规则、数据库、网络或测试数据

课堂判断：这段代码为什么难以写单元测试？

正面代码：依赖注入让外部依赖可替换

依赖注入不是形式主义，它把测试环境从真实外部系统中解放出来。

```
class ReservationService:
    def __init__(self, device_repo: DeviceRepository,
                 notification_client: NotificationClient):
        self.device_repo = device_repo
        self.notification_client = notification_client
```

接口边界

DeviceRepository 与 NotificationClient 成为可替换依赖

Mock 替身

测试时不启动真实 DB 和通知服务

快速定位

失败基本指向 ReservationService 业务逻辑

名词解释/简答：依赖注入如何提升可测试性?

同一功能，两种架构的测试成本差异

架构选择会把测试成本放大或缩小，差异往往不是 10% 而是数量级。

耦合写法

启动 DB、启动通知服务、准备全量测试数据，失败定位粗

→

依赖注入

Mock 替换外部依赖，毫秒级运行，失败定位到业务逻辑

工程后果

CI 是否跑得动，团队是否信任测试，取决于这类结构选择

可测试性差的五种典型架构特征

当系统必须整体启动才能验证局部逻辑时，测试成本已经被架构锁死。

直接访问数据库

测试依赖真实表结构和数据

硬编码外部调用

网络和第三方服务成为测试前提

逻辑混在 Controller

只能通过 HTTP 测业务规则

全局状态/单例

测试之间互相污染

测试金字塔：不是背图，而是资源配置

越靠底层的测试越快越便宜，越靠顶层的测试越真实但越慢越难定位。

端到端测试
少量 | 完整链路 | 最真实但最慢

集成测试
适量 | 模块协作 | 数据库/接口/消息

单元测试
大量 | 业务规则 | 快速精确定位

工程含义

预约系统不应把所有校验逻辑都放到 UI 自动化里测。

判断规则

名词解释：测试金字塔的工程含义。

测试层级 × 架构层对应关系

测试层级选择，本质上是选择在哪个架构层验证哪类风险。

单元测试	验证：函数/类逻辑；架构层：领域模型层；速度/成本：毫秒级、极低；定位：精确到函数。
集成测试	验证：模块协作、接口契约；架构层：服务边界、数据访问层；速度/成本：秒级、中等；定位：模块间。
端到端测试	验证：完整业务链路；架构层：整个系统 + 外部依赖；速度/成本：分钟级、高；定位：较难。

本页规则

50 并发预约需要真实数据库，通常属于集成测试而不是单元测试。

考试锚点

综合题：给定质量场景，选择测试层级并说明理由。

单元测试：把业务规则单独验证清楚

单元测试适合验证纯逻辑、边界值和状态机，不适合验证外部系统协作。

适合

预约时长 1-4 小时、资格校验、状态机转换

不适合

通知是否真实发送、数据库是否持久化、前端表单是否可用

关键

让规则从 Controller 和数据库中抽出来，成为可单独验证的对象

案例

“预约时长必须是 1-4 小时整数”应写单元测试。

课堂锚点

选择题：哪些场景应该用单元测试？

集成测试：验证模块接起来以后是否正确

集成测试关注边界协作，尤其适合数据库、消息队列、接口契约和权限链。

模块协作

预约服务 + 设备库存：提交后设备状态是否占用

数据行为

同一设备同一时段并发写入是否只有一条有效记录

外部替代

可用内存数据库、测试容器或 Mock Server 替代真实依赖

案例

JMeter 50 并发 POST 同一 device_id 和 timeslot。

课堂锚点

判断题：并发一致性为什么通常不是单元测试？

端到端测试：只保留最高价值的完整链路

端到端测试最接近真实用户，但数量必须克制，否则 CI 会失去可用性。

适合

学生登录 → 查看设备 → 提交预约 → 收到通知

成本

运行慢、环境依赖多、失败定位粗

原则

覆盖主流程和高风险路径，不覆盖所有边界值

案例

端到端测试应验证核心 Happy Path，而不是把所有表单边界都放进去。

课堂锚点

简答题：端到端测试为什么不能过多？

失败案例：所有测试都压在端到端

当底层逻辑不可测时，团队会被迫用慢而脆的端到端测试弥补架构缺口。

现象

80% 自动化测试都是 UI E2E，CI 每次 35 分钟

后果

失败无法定位，flaky tests 增多，团队开始无视红灯

根因

业务逻辑散落在 Controller，无法写有效单元测试

案例

Selenium 测试堆积导致自动化测试形同虚设。

课堂锚点

案例分析：表面原因和深层架构原因分别是什么？

课堂判断：四个场景分别放在哪一层测？

选测试层级时，不看测试工具名，而看需要验证的架构层。

A

预约时长必须是 1-4 小时整数

B

审批通过后通知服务能收到消息

C

学生从页面提交预约到收到通知邮件

D

50 个并发预约同一设备，只有 1 人成功

质量门禁：进入下一阶段的可量化标准

质量门禁把“差不多可以”变成“满足这些条件才允许继续”。

定义

构建或版本进入下一阶段前必须满足的一组可量化通过标准

价值

让质量从主观判断变成工程约束

要求

自动化、可检查、可阻断、不可随意跳过

案例

“基本没问题”不是门禁；“P0=0、E2E核心路径100%通过”才是门禁。

课堂锚点

名词解释：质量门禁 Quality Gate。

质量门禁

三层质量门禁：代码层、构建层、发布层

质量门禁要分层设置，越往后代价越高，越不能把问题留到最后。

代码层

单元测试、静态分析、覆盖率、代码评审

构建层

BVT、集成测试、性能基线、安全依赖扫描

发布层

核心 E2E、回归测试、负载测试、P0/P1 缺陷清零

案例

预约冲突并发测试不应等发布前才第一次运行。

课堂锚点

简答题：三层质量门禁分别检查什么？

质量门禁

预约系统 M4 质量门禁示例

好的门禁清单必须写出必须项、推荐项和明确数值。

必须满足

- 创建/审批/取消核心集成测试全部通过
- 50 并发同一设备，201=1 条、409=49 条、DB 记录=1 条
- 学生 Token 访问 /admin/** 返回 403

推荐满足

- 预约服务域覆盖率 ≥ 65%，设备状态机 ≥ 80%

课堂任务：把本组门禁写成可勾选清单。

CI/CD 中的门禁位置

门禁只有嵌入流水线，才会从口头约定变成工程事实。

提交
开发提交代码

→

代码门禁
单元+静态+覆盖率

→

合并
进入主分支

→

构建门禁
BVT+集成+性能

→

测试环境
预发布验证

→

发布门禁
E2E+回归+缺陷

工程约束

紧急发布也不能跳过权限和并发一致性门禁。

课堂判断

判断题：质量门禁能否临时跳过？说明风险。

坏门禁与好门禁的差别

门禁是否有效，取决于标准能否被自动或人工一致地判断。

坏标准

性能可接受、基本没有严重缺陷、测试基本通过

→

好标准

P95 ≤ 500ms、P0=0、核心集成测试通过率=100%

判断规则

能量化、能复现、能阻断、责任清楚

契约测试前先问：有几个独立部署服务？

契约测试不是所有项目都需要，它首先取决于部署形态。

适用

前后端或微服务独立部署，接口变化会影响其他团队

不适用

模块化单体内部模块，不需要引入 Pact 复杂度

替代

单体系统用接口测试或集成测试验证边界

判断动作

判断题：本组系统是否需要契约测试？为什么？

契约测试：消费方与提供方共同验证接口约定

契约测试解决的是独立部署服务之间“接口各自改坏而不自知”的问题。

消费方

声明自己期望的请求、响应结构和状态码

契约仓库

保存接口期望，供提供方 CI 拉取验证

提供方

验证自己的实现仍满足消费方契约

案例

前端期待 /api/devices 返回 devices 列表，后端改字段名会被 CI 发现。

课堂锚点

名词解释：契约测试 Contract Testing。

Pact 工作流：接口变化在双方 CI 中暴露

契约测试把接口不兼容从端到端阶段提前到各自构建阶段。

1 消费方定义
前端测试描述期望接口

→

2 发布契约
Pact 文件进入 Broker

→

3 提供方验证
后端 CI 拉取契约并验证 API

→

4 失败阻断
字段、状态码或结构不匹配时构建失败

案例
后端把 status 改成 state，前端契约验证失败。

考试锚点
简答题：契约测试为什么比端到端测试更早发现接口问题？

模块化单体：用接口测试替代契约测试

技术选择要服务复杂度，单体内部边界用集成接口测试通常更合适。

模块化单体

所有模块同进程，接口变化可在同一代码库内验证

→

测试方式

启动 Spring 容器 + 内存数据库，验证 HTTP 状态码和响应结构

判断

不要为了“现代”而引入与团队规模不匹配的测试体系

安全是高损失质量属性

安全测试不能等功能完成后补，因为很多漏洞的根在架构层。

高损失

越权、注入、认证绕过可能直接导致数据泄露

架构层

权限、认证、数据访问、传输加密都有明确位置

验证层

安全测试要对应到具体架构层，而不是泛泛扫漏洞

案例

学生访问管理员预约列表返回 200，是架构权限层缺失。

课堂锚点

简答题：为什么安全问题是架构问题？

安全质量

越权访问：权限层必须集中处理

授权不能依赖每个 Controller 自觉检查，架构上应有统一权限拦截层。

风险

学生直接请求 /api/admin/reservations 获取全部预约记录

架构做法

Security Filter、AOP 或 @PreAuthorize 集中拦截

测试方式

学生 Token 请求管理员接口，必须返回 403 且无敏感数据

案例题：设计越权访问测试用例。

SQL 注入：数据访问层要消除拼接 SQL

注入漏洞不是靠上线前扫一下解决，而是靠数据访问层的参数化约束预防。

风险

device_id 输入 1 OR 1=1，查询返回不该返回的数据

架构做法

ORM、参数化查询、统一 Repository 规范

测试方式

恶意参数应返回 400 或空结果，不能返回全部设备

名词解释/简答：SQL 注入与参数化查询。

安全质量

认证绕过：Token 校验必须覆盖过期与伪造

认证测试要验证完整过滤链，而不是只测登录函数能否返回 Token。

风险

过期 JWT 仍能访问预约接口

架构做法

认证 Filter 统一校验签名、过期时间和用户状态

测试方式

过期 Token 请求 /api/reservations，必须返回 401

判断题：认证绕过测试为什么通常是集成测试？

安全问题、架构层与测试层对应表

安全测试要能指出问题属于哪一层，否则修复会变成到处打补丁。

越权访问

权限过滤层/AOP，集成测试，修复难度高

SQL 注入

数据访问层，单元/集成测试，参数化查询

认证绕过

API 网关/Filter，集成测试，401 验证

敏感数据明文

存储/传输层，审查+集成测试，加密改造

质量闭环

缺陷生命周期：从发现到复盘

缺陷关闭不是终点，复盘和补测试才让缺陷转化为质量改进。

发现 → 复现 → 定级 → 修复 → 补测试 → 验证 → 关闭 → 复盘

关键

补测试用例，防止同类问题再次进入主分支

质量数据

缺陷分布能暴露脆弱模块和技术债集中区

质量闭环

规范缺陷报告：并发预约重复占用

好的缺陷报告要让开发者能复现、能判断优先级、能补回归测试。

重现步骤

JMeter 20 线程同时 POST 相同 device_id 与 timeslot

期望结果

1 个 201，19 个 409，数据库有效记录 = 1

实际结果

3-5 个 201，出现多条有效预约记录

分析

缺少乐观锁/悲观锁，多个事务同时通过可用性检查

质量闭环

缺陷是质量数据，不只是 Issue 清单

缺陷积累起来能指出架构薄弱点、测试盲区和团队流程问题。

模块脆弱性

同一服务反复出现缺陷，说明边界或职责可能不清

测试盲区

同类缺陷多次逃逸，说明门禁没有覆盖

技术债

修复成本越来越高，说明结构已经阻碍变更

案例

权限相关 bug 反复出现，往往说明缺少统一权限层。

课堂锚点

简答题：缺陷数据能反映哪些架构问题？

AI 能辅助测试什么

AI 适合做测试素材生成和变体枚举，不适合替代测试策略判断。

能做

生成测试骨架、枚举边界值、写 Mock/Stub、整理安全清单

能帮

解释报错、补充测试数据变体、把规则转成断言模板

前提

人必须先说明业务规则、架构层级和风险优先级

案例

让 AI 枚举预约时长边界：0、1、4、5、负数、小数、null。

课堂锚点

判断题：哪些测试工作适合交给 AI 辅助？

AI 不能替你决定测试策略

AI 生成再多测试，如果层级选错，关键风险仍然会逃逸。

不能替代

测试层级选择、业务规则判定、风险优先级排序

典型误区

100 个单元测试全绿，但并发数据库问题仍未发现

人的职责

先识别风险，再决定层级，最后让 AI 辅助生成骨架

AI 辅助测试的正确工作流

先由人做策略判断，再让 AI 做低风险的生成和补全工作。

你做

识别风险、决定测试层级、写清业务期望

AI做

生成测试函数骨架、Mock 代码、边界值变体

你确认

运行验证、补充断言、筛选值得保留的用例

案例

并发预约：人决定集成测试，AI 辅助生成 JMeter/pytest 数据模板。

课堂锚点

简答题：AI 辅助测试的边界是什么？

课堂演练：为质量属性场景补“测试方案”列

测试方案要写清测试层级、测试方式和可量化通过标准。

输入

第4讲质量属性场景表：刺激源、环境、响应、可量化指标

新增列

测试层级 / 测试方式 / 通过标准

目标

每组完成 3-5 条关键质量属性场景的测试方案

课堂产出：测试方案列可直接进入 M4 测试计划。

课堂演练

演练示例：50 并发预约同一设备

通过标准必须能被一致判断，不能写成“表现正常”这类模糊表述。

测试层级	集成测试：需要真实数据库，不需要浏览器
测试方式	JMeter 50 并发 POST /api/reservations，相同设备与时段
通过标准	201=1 条，409=49 条，DB 中该时段有效记录=1 条

考试锚点：把并发质量属性写成测试方案。

课堂演练

互审标准：层级选对了吗？标准可量化吗？

互审时优先看两件事：测试层级是否合理，通过标准是否可判定。

层级检查

纯逻辑不要放端到端；并发数据库不要放单元测试

标准检查

避免“可接受”“基本正常”“没有明显错误”

建议写法

P95≤2000ms、错误率≤0.1%、重复预约数=0

案例

邻组在问题行旁写一条修改建议。

课堂锚点

课堂动作：交换表格，标出 2 处可改进项。

M4 产出一：测试计划 V1

测试计划要说明风险如何被覆盖，而不是简单罗列测试工具。

测试范围

本迭代覆盖哪些核心功能，不覆盖什么及原因

层级分配

单元、集成、端到端分别覆盖哪些模块和场景

测试环境

本地、内存数据库、Docker Compose、E2E 环境

关键风险

并发一致性、通知降级、越权访问的测试方案

M4 产出二：质量门禁清单

门禁清单要能被评审者逐项勾选，不满足就应说明原因和处理计划。

代码层

单元测试通过率=100%，新增代码无 Blocker 告警

集成层

核心业务流程集成测试全部通过，关键并发场景通过

发布层

P0 缺陷=0，越权测试通过，P95 响应时间达标

连接

下一讲补充发布运维方案 V1，合并形成 M4

本讲七条关键判断

如果测试难写，优先回头检查架构边界、依赖隔离和质量门禁。

架构与测试

- 1. 可测试性由架构决定，不是测试工具决定
- 2. 测试金字塔对应架构层，选错层测再多也没用
- 3. 质量门禁是工程约束，不是主观判断
- 4. 契约测试只适合独立部署服务，单体用接口测试替代

工程与交付

- 5. 安全问题要定位到权限、认证、数据访问等架构层
- 6. AI 能生成骨架，但测试策略必须由人决定
- 7. 测试难写通常暴露了架构耦合点